🎓 DBMS Course Project

Course: Database Management Systems (DBMS)

- Task 1 : Salina, Jyoti ,Rahul , Dil
- Task 2 : Sisan, Anuj , Sohan,
- Task 3 : Saurav, Saugat , Sharad

Prepare a report , presentation and demo and present in class .

Date of presentation : **Tuesday Jun2 , 2026 .**

📚 Task 1: Library Management System

Your Assigned Table Structures:

- Student columns: (StudentID, Name, Email, Department, AccountStatus)
- Book columns: (BookID, Title, Author, PublishYear, AvailableCopies)
- Loan columns: (LoanID, BookID, StudentID, IssueDate, DueDate, ReturnStatus)
- Fine columns: (FineID, LoanID, Amount, PaymentStatus)

Your Project Tasks:

1. Entity-Relationship (ER) Diagram

- **Task:** Draw a visual ER Diagram mapping out the 4 assigned tables. You must decide which columns are the **Primary Keys (PK)**. Show how the tables connect using lines, and clearly mark the relationship types (such as 1-to-Many).

2. Creating Tables & Referential Integrity

- **Task:** Write the SQL CREATE TABLE commands for all 4 tables. You must explicitly declare your chosen **Primary Keys**. You must also link the tables together using **Foreign Keys (FK)** based on the columns that match.
- **Validation Rule:** Ensure that if a student profile is deleted from the Student table, all of their history in the Loan and Fine tables is automatically removed (ON DELETE CASCADE).

3. Domain Constraints

- **Task:** Add safety rules (CHECK constraints) to your table creation code to protect data integrity:

    - **Validation Rule 1:** The AvailableCopies inside the Book table can never drop below 0.
    - **Validation Rule 2:** The penalty Amount inside the Fine table can never be a negative number.

4. Inserting & Updating Data (DML)

- **Task:** Write INSERT INTO statements to add at least **4 realistic rows** of sample data into each table.
- **Validation Rule:** Write an UPDATE statement that automatically finds all rows in the Fine table marked as 'Unpaid' and increases their Amount by 10%.

5. Basic Select Queries

- **Task:** Write a query using SELECT... FROM... WHERE along with text filtering operators.
    - **Validation Rule:** Retrieve only the rows from the Book table where the title contains the word 'Data' **AND** the PublishYear is after 2018.

6. Aggregate Functions (GROUP BY & HAVING)

- **Task:** Write a query that calculates statistical summaries for grouped data.
    - **Validation Rule:** Group your data by StudentID, count their total transactions using COUNT(LoanID), and display only the rows for students who have borrowed more than 2 books in total.

7. Subqueries

- **Task:** Write a query that embeds an inner query inside an outer query (NOT IN).
    - **Validation Rule:** Find "dead inventory" by listing the titles of all books from the Book table whose IDs do not exist anywhere inside the Loan table.

8. Database Trigger

- **Task:** Create an automated TRIGGER program inside your database.
    - **Validation Rule:** The moment a user inserts a brand new row into the Loan table (simulating a book check-out), the trigger must instantly update the Book table and subtract 1 from that book's AvailableCopies.

9. Global Assertion

- **Task:** Write a declarative database rule using standard CREATE ASSERTION syntax.
    - **Validation Rule:** Enforce a strict global limit ensuring that no single student identity can have more than 3 active items in the Loan table at the exact same time.

10. Access Management (Security)

- **Task:** Write the database administration script to secure system data:
    - Create two distinct roles: `AppAdmin` and `RegularUser`.
    - Use `GRANT` to give the admin editing rights, but give the user read-only (`SELECT`) rights.
    - Use `REVOKE` to strip away the `DELETE` privilege from the `AppAdmin` role to prevent accidental data erasure.

🍔 Task 2: Canteen Management System

Your Assigned Table Structures:

- `Customer` columns: (`CustomerID`, `Name`, `Phone`, `AccountType`, `Status`)
- `MenuItem` columns: (`ItemID`, `ItemName`, `Category`, `Price`, `StockQuantity`)
- `FoodOrder` columns: (`OrderID`, `CustomerID`, `OrderDate`, `TotalAmount`, `PaymentStatus`)
- `OrderLine` columns: (`LineID`, `OrderID`, `ItemID`, `QuantityOrdered`, `SubTotal`)

Your Project Tasks:

1. Entity-Relationship (ER) Diagram

- **Task:** Draw a visual ER Diagram mapping out the 4 assigned tables. You must decide which columns are the **Primary Keys (PK)**. Show how the tables connect using lines, and clearly mark the relationship types (such as 1-to-Many).

2. Creating Tables & Referential Integrity

- **Task:** Write the SQL `CREATE TABLE` commands for all 4 tables. You must explicitly declare your chosen **Primary Keys**. You must also link the tables together using **Foreign Keys (FK)** based on the columns that match.
- **Validation Rule:** Ensure that if a customer account is deleted from the `Customer` table, all of their transaction logs inside the `FoodOrder` table are automatically cleared out (`ON DELETE CASCADE`).

3. Domain Constraints

- **Task:** Add safety rules (`CHECK` constraints) to your table creation code to protect data integrity:
    - **Validation Rule 1:** The `Price` of a dish inside the `MenuItem` table must always be greater than 0.
    - **Validation Rule 2:** The `QuantityOrdered` inside the `OrderLine` table can never be 0 or negative.

4. Inserting & Updating Data (DML)

- **Task:** Write INSERT INTO statements to add at least **4 realistic rows** of sample data into each table.
- **Validation Rule:** Write an UPDATE statement that targets the MenuItem table and increases the Price by 10% for all items belonging to the 'Snacks' category.

5. Basic Select Queries

- **Task:** Write a query using SELECT... FROM... WHERE along with text filtering operators.
    - **Validation Rule:** Retrieve only the rows from the MenuItem table where the category is exactly 'Drinks' **AND** the Price is less than 3.00.

6. Aggregate Functions (GROUP BY & HAVING)

- **Task:** Write a query that calculates statistical summaries for grouped data.
    - **Validation Rule:** Group your orders by CustomerID, calculate their total money spent using SUM(TotalAmount), and display only the names of customers who have spent a grand total of more than 40.00.

7. Subqueries

- **Task:** Write a query that embeds an inner query inside an outer query (NOT IN).
    - **Validation Rule:** Identify unpopular dishes by listing the names of all food items from the MenuItem table that have never been recorded inside the OrderLine table.

8. Database Trigger

- **Task:** Create an automated TRIGGER program inside your database.
    - **Validation Rule:** The moment a row is inserted into the OrderLine table (simulating an item being purchased), the trigger must instantly update the MenuItem table and subtract the QuantityOrdered from that item's StockQuantity.

9. Global Assertion

- **Task:** Write a declarative database rule using standard CREATE ASSERTION syntax.
    - **Validation Rule:** Enforce a strict global credit policy ensuring that no customer can place a new order if their cumulative unpaid amount across the system exceeds 50.00.

10. Access Management (Security)

- **Task:** Write the database administration script to secure system data:
    - Create two distinct roles: AppAdmin and RegularUser.
    - Use GRANT to give the admin editing rights, but give the user read-only (SELECT) rights.

    - Use REVOKE to strip away the DELETE privilege from the AppAdmin role to prevent accidental data erasure.

💻 Task 3: Assignment Management Portal

Your Assigned Table Structures:

- Student columns: (StudentID, FullName, Email, Major, SubmissionsCount)
- Course columns: (CourseID, CourseName, Department, Credits)
- Assignment columns: (AssignmentID, CourseID, Title, MaxMarks, PublishDate)
- Submission columns: (SubmissionID, AssignmentID, StudentID, SubmissionDate, Grade)

Your Project Tasks:

1. Entity-Relationship (ER) Diagram

- **Task:** Draw a visual ER Diagram mapping out the 4 assigned tables. You must decide which columns are the **Primary Keys (PK)**. Show how the tables connect using lines, and clearly mark the relationship types (such as 1-to-Many).

2. Creating Tables & Referential Integrity

- **Task:** Write the SQL CREATE TABLE commands for all 4 tables. You must explicitly declare your chosen **Primary Keys**. You must also link the tables together using **Foreign Keys (FK)** based on the columns that match.
- **Validation Rule:** Ensure that if a subject is deleted from the Course table, all associated assignments and student records inside the Assignment and Submission tables are automatically wiped (ON DELETE CASCADE).

3. Domain Constraints

- **Task:** Add safety rules (CHECK constraints) to your table creation code to protect data integrity:
    - **Validation Rule 1:** The MaxMarks listed inside the Assignment table must always be a positive value greater than 0.
    - **Validation Rule 2:** The Grade awarded to a student inside the Submission table can never be recorded as a negative number.

4. Inserting & Updating Data (DML)

- **Task:** Write INSERT INTO statements to add at least **4 realistic rows** of sample data into each table.
- **Validation Rule:** Write an UPDATE statement that targets the Submission table and adds 5 bonus marks to the Grade column for all files uploaded before a specific deadline date.

5. Basic Select Queries

- **Task:** Write a query using SELECT... FROM... WHERE along with text filtering operators.
    - **Validation Rule:** Retrieve only the rows from the Assignment table where the title contains the word 'Project' **AND** the MaxMarks value is greater than 20.

6. Aggregate Functions (GROUP BY & HAVING)

- **Task:** Write a query that calculates statistical summaries for grouped data.
    - **Validation Rule:** Group your submissions by CourseID, calculate the class average using AVG(Grade), and display only the IDs of struggling courses where the class performance average falls below 60%.

7. Subqueries

- **Task:** Write a query that embeds an inner query inside an outer query (NOT IN).
    - **Validation Rule:** Track down missing work by selecting the names of all students from the Student table who have never submitted any files to the Submission table.

8. Database Trigger

- **Task:** Create an automated TRIGGER program inside your database.
    - **Validation Rule:** The moment a student successfully inserts a new row into the Submission table, the trigger must instantly update the Student table and increment their SubmissionsCount column by exactly 1.

9. Global Assertion

- **Task:** Write a declarative database rule using standard CREATE ASSERTION syntax.
    - **Validation Rule:** Enforce a strict chronological timeline rule checking that an assignment's final deadline date can never be saved as a calendar date that occurs *before* its own creation or publish date.

10. Access Management (Security)

- **Task:** Write the database administration script to secure system data:
    - Create two distinct roles: AppAdmin and RegularUser.
    - Use GRANT to give the admin editing rights, but give the user read-only (SELECT) rights.
    - Use REVOKE to strip away the DELETE privilege from the AppAdmin role to prevent accidental data erasure.